PARC 2026 – Topics de l'arbitre

Votre solution communique avec l'arbitre par cinq topics sous `/parc/`. Les types de messages seront fournis dans le paquet `parc_msgs`.

Topics

Topic	Sens	Type	QoS	Quand
`/parc/tag_order`	Arbitre → vous	`parc_msgs/TagOrder`	Reliable, transient local, profondeur 1	Quand les juges arment votre course
`/parc/start`	Vous → arbitre	`parc_msgs/StartSignal`	Reliable, profondeur 10	Une fois, au départ du robot
`/parc/tag_visit`	Vous → arbitre	`parc_msgs/TagVisit`	Reliable, profondeur 10	Une fois par tag, robot arrêté en position de visite
`/parc/finish`	Vous → arbitre	`parc_msgs/Finish`	Reliable, profondeur 10	Uniquement pour terminer la course plus tôt
`/parc/run_status`	Arbitre → vous	`parc_msgs/RunStatus`	Reliable, transient local, profondeur 1	2 fois par seconde et à chaque changement

Ne publiez sur aucun autre topic `/parc/`, ni sur les topics des pilotes du robot, comme l'odométrie des roues.

Messages

```
# parc_msgs/msg/TagOrder.msg
builtin_interfaces/Time stamp
uint8[] tag_ids            # p. ex. [7, 1, 9, 4]
```

```
# parc_msgs/msg/StartSignal.msg
builtin_interfaces/Time stamp
uint8 METHOD_COMMAND = 0   # départ par commande
uint8 METHOD_VOICE   = 1   # mot de départ (bonus)
uint8 method
string heard               # mot reconnu : go, allez ou vai

# parc_msgs/msg/TagVisit.msg
std_msgs/Header header     # stamp : heure, frame_id : caméra
uint8 tag_id               # 1, 4, 7 ou 9
# JPEG où le tag est bien visible :
sensor_msgs/CompressedImage image

# parc_msgs/msg/Finish.msg
builtin_interfaces/Time stamp
uint8 REASON_COMPLETED = 0
uint8 REASON_GAVE_UP   = 1
uint8 reason

# parc_msgs/msg/RunStatus.msg
builtin_interfaces/Time stamp
uint8 IDLE       = 0       # aucune course en cours
uint8 ARMED      = 1       # ordre envoyé, attente du départ
uint8 RUNNING    = 2       # chronomètre en marche
uint8 FINISHED   = 3       # dernier tag, finish ou juge
uint8 TIMED_OUT  = 4       # 10 minutes atteintes
uint8 TERMINATED = 5       # arrêt de sécurité par un juge
uint8 state
float32 elapsed_s
float32 time_limit_s       # 600
uint8[] tag_order
uint8[] tags_reported      # reçues, pas encore jugées
uint8 start_method         # StartSignal.METHOD_*
bool false_start           # mouvement avant le départ
```

Pendant une course

1. **Avant le départ :** lancez votre solution et abonnez-vous à `/parc/tag_order` et `/parc/run_status`. Quand les juges arment votre course, l'ordre arrive et l'état passe à `ARMED`. Le robot ne doit pas encore bouger : tout mouvement est signalé comme un faux départ.
2. **Départ :** publiez un seul `StartSignal`, avec `method` à `METHOD_VOICE` et `heard` renseigné si le mot de départ a été utilisé. Le chronomètre démarre à sa réception par l'arbitre, et l'état passe à `RUNNING`.

3. **À chaque tag :** arrêtez-vous en position de visite, publiez un seul `TagVisit` avec une image du tag, restez immobile au moins 3 secondes, puis repartez. Signalez chaque tag une seule fois.
4. **Fin :** la course se termine automatiquement lorsque vous signalez le dernier tag de l'ordre. Si votre robot saute des tags et ne peut pas signaler le dernier, publiez `Finish` pour arrêter le chronomètre.
5. **Arrêt :** lorsque `/parc/run_status` indique `FINISHED`, `TIMED_OUT` ou `TERMINATED`, arrêtez le robot.

`tags_reported` dans `/parc/run_status` confirme la réception de chaque visite. Ce sont les juges qui décident si une visite rapporte des points.

Exemple (Python)

```
from rclpy.qos import (QoSProfile, ReliabilityPolicy,
                       DurabilityPolicy)
from parc_msgs.msg import (TagOrder, StartSignal, TagVisit,
                           Finish, RunStatus)

# Pour /parc/tag_order et /parc/run_status :
LATCHED = QoSProfile(
    depth=1, reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL)
# Pour /parc/start, /parc/tag_visit et /parc/finish :
EVENTS = QoSProfile(depth=10,
                    reliability=ReliabilityPolicy.RELIABLE)

# Dans le __init__ de votre nœud :
self.create_subscription(TagOrder, '/parc/tag_order',
                         self.on_order, LATCHED)
self.create_subscription(RunStatus, '/parc/run_status',
                         self.on_status, LATCHED)
self.start_pub = self.create_publisher(
    StartSignal, '/parc/start', EVENTS)
self.visit_pub = self.create_publisher(
    TagVisit, '/parc/tag_visit', EVENTS)
self.finish_pub = self.create_publisher(
    Finish, '/parc/finish', EVENTS)

# Départ (commande) :
msg = StartSignal()
msg.stamp = self.get_clock().now().to_msg()
msg.method = StartSignal.METHOD_COMMAND
self.start_pub.publish(msg)
```

```
# À un tag, robot arrêté en position de visite.
# image est un sensor_msgs/CompressedImage (JPEG).
msg = TagVisit()
msg.header.stamp = self.get_clock().now().to_msg()
msg.header.frame_id = image.header.frame_id
msg.tag_id = 7
msg.image = image
self.visit_pub.publish(msg)
# Puis restez immobile au moins 3 secondes.
```